SUBMISSION ITEM 05

Security self-assessment report

Permission tiers and known risks

Who can do what in Boundra, where each limit is enforced, and the risks that remain, with their mitigations.

Event	FinTechathon 2026, Topic F
Network	Base Sepolia testnet (chain 84532), MockUSDC and mock vaults
Status	Self-assessment, not an independent audit
Date	20 October 2026

1 Summary

Boundra gives an AI agent a wallet it cannot misuse. Authority is split into tiers: only the *owner* can grant, change or end authority and take funds back; the *agent* can only make two typed calls, inside limits the owner signed; everyone else (risk admin, backend, apps holding an API token, the language model) has narrower powers still. The decisive limits are enforced by the owner's executor contract on chain, not by the backend's good behaviour.

The guarantee

Even if the agent's key is stolen, the server is compromised or the model is wrong, the worst outcome is bounded by the mandate the owner signed: capped amounts, one allocation, allowed vaults only, withdrawals paid to the owner, and the owner can pause or revoke at any time.

Scope. This is the team's self-assessment of a testnet prototype (Base Sepolia, MockUSDC, two controlled mock vaults). It is not an independent audit and not an approval for real funds.

2 Permission tiers

2.1 The tiers

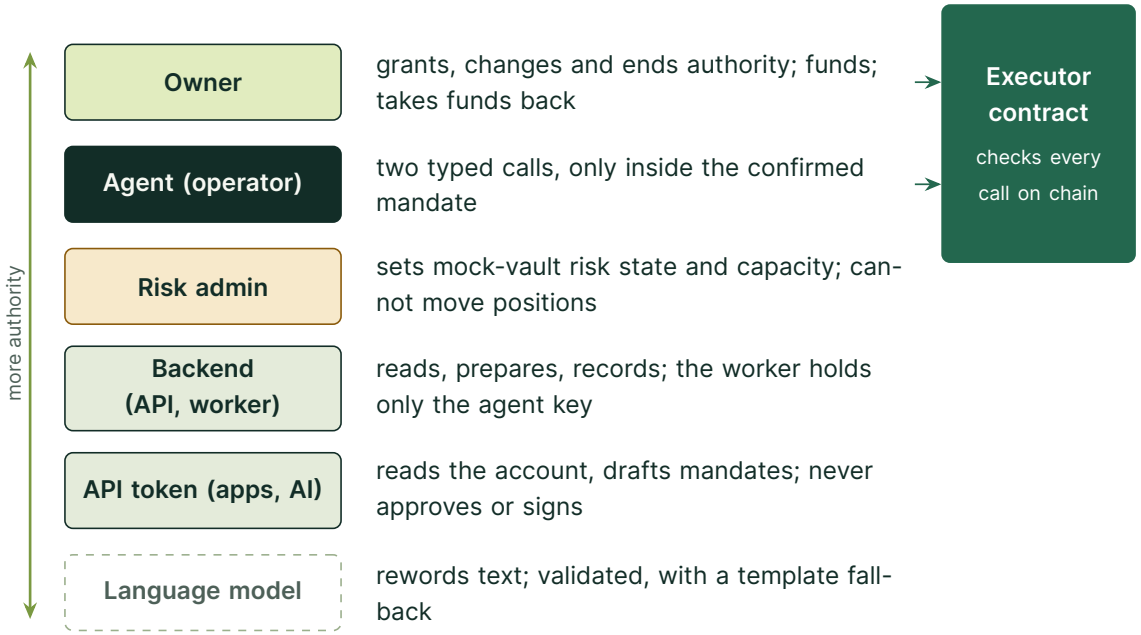


Figure 1: From most to least authority. Only the owner and the agent can change anything on chain, and the executor checks both.

2.2 Permission matrix

Action	Owner	Agent	Risk admin	Backend	API token	Model
Create executor, set mandate, fund	**yes**	no	no	no	no	no
Pause, resume, revoke	**yes**	no	no	no	no	no
Recover idle funds, withdraw a position to the owner	**yes**	no	no	no	no	no
`executeDeposit` / `executeWithdraw` within the mandate	no	**yes**	no	no	no	no
Send funds anywhere but the owner	no	no	no	no	no	no
Set a vault's risk state and capacity	no	no	**yes**	no	no	no
Approve a mandate draft (preview hash)	**yes**	no	no	no	no	no
Start a goal, answer questions, draft a mandate	**yes**	no	no	no	**yes**	no
Read the dashboard and activity	**yes**	no	no	no	**yes**	no
Mark a mandate `AUTHORIZED` (after chain checks)	no	no	no	**yes**	no	no
Choose questions, vault or limits	no	no	no	**yes**[*]	no	no
Reword a question or summary	no	no	no	no	no	**yes**

[*] Deterministic code (ACCE and the mandate builder); the owner reviews and approves the result.

2.3 How each tier is implemented

Tier	Enforced by	Where
Owner	`onlyOwner` on every authority and recovery function; the owner is immutable per executor; withdrawals always pay the owner	`contracts/src/BoundraExecutor.sol`
Agent	Operator address, current version and nonce, fresh action ID, exclusive deadline, allowed vault and action, per-action and total caps, one allocation, lockup and expiry, no deposit into a degraded vault	`BoundraExecutor.sol` (`_authorize`); checked again off chain by `prepareAction` and simulation
Risk admin	Vault owner only; changes risk flag and global capacity, never a position	`contracts/src/MockVault.sol`

Tier	Enforced by	Where
Backend	No owner keys; worker modes (`observe_only`, `monitor_only`, `execute`); refuses an operator equal to the risk admin; reviewed manifest with pinned bytecode hashes	`apps/worker/src/runtime/activation.ts`, `packages/chain`
Browser session	SIWE bound to origin and chain, single-use nonce, `__Host-` cookie (HttpOnly, Secure, SameSite=Strict), exact `Origin` and CSRF token on every change; owner always from the session	`apps/api/src/auth`
API token	Signed token statement decides the session kind; bearer only, never a cookie; admin routes refused; hashed, expiring, revocable from the app	`apps/api/src/auth`, `/app/connect`
Admin	Wallets in `ADMIN_ADDRESSES`, browser session only	`apps/api/src/auth`
Language model	Output schema, same alternatives, no new numbers, vaults, addresses, links or guarantees; time and token budgets; template fallback	`packages/llm`

Defence in depth. Every agent action passes four gates: the deterministic policy, the off-chain chain guard (`prepareAction`), a simulation against current state, and the executor's own on-chain checks. Only the last one is trusted; the others fail early and explain.

3 Known-risk list

Impact is rated for this testnet prototype; "real funds" marks risks that would be blocking before any mainnet use.

ID	Risk	Impact	Mitigation in place	Residual risk
R1	Agent key stolen	**High**	Bounded by the mandate: caps, one allocation, allowed vaults, withdrawals only to the owner. Separate account; owner can pause or revoke and recover funds.	Remaining budget can be used at an inconvenient time.
R2	Server or hosting compromised	**High**	No owner keys on the server; secrets are runtime settings only; agent key bounded as in R1; API tokens cannot move funds.	Misleading drafts are possible; the owner reviews the exact preview before approving.
R3	Wrong agent decision	**Medium**	Deterministic, tested policy; `prepareAction`, simulation and on-chain checks.	Caps limit harm but do not prove suitability.

ID	Risk	Impact	Mitigation in place	Residual risk
R4	Look-alike contracts or fake events	**High**	Reviewed manifest with pinned runtime-bytecode hashes; risk events only from the two pinned vaults.	The manifest review itself is manual.
R5	RPC provider fails or misreports	**Medium**	Confirmation depth, block-hash checks, receipt re-checks; on failure the worker holds instead of acting.	A single public RPC is trusted; no second source.
R6	Chain reorganisation	**Low**	5-block depth, 20-block scan overlap, terminal receipts re-checked for 20 blocks.	A deeper reorg forces a rescan.
R7	Stuck or duplicated agent transaction	**Medium**	Persist before broadcast, fenced leases, idempotent nonce reservations, identical-byte recovery, release after the deadline.	No automatic fee bumping.
R8	Risk admin is a central fixture	**Medium**	Can only set risk state and capacity; separate account from operator and deployer.	Not an oracle; stands in for real protocol signals.
R9	Phishing or cross-site requests	**Medium**	Origin-bound SIWE, `__Host-` cookie, exact Origin and CSRF checks; wallet prompts show the real call.	Owners must still check the site address.
R10	API token leaked	**Medium**	Cannot approve, sign or move funds; hashed, expiring and revocable.	Can read the account and draft mandates until revoked.
R11	Language model misleads	**Low**	Wording only; validated; template fallback; labelled source.	None for authority.
R12	Mock vaults and MockUSDC	**Low**	Testnet only; principal 1:1; yields are labelled illustrative.	Real vault integrations need their own review (real funds).
R13	No independent audit	**High**	Unit, fuzz and stateful invariant tests; end-to-end runs.	Blocking before real funds.
R14	Free hosting sleeps	**Medium**	Health endpoints; the container restarts if any process stops.	The agent sleeps with the service; a paid plan or VPS removes this.
R15	Stale screen invited repeated owner actions	**Low**	Repeats were harmless on chain; the fix polls until the confirmed block and disables the controls meanwhile (PR #11).	None once the fix is deployed.
R16	Dependency supply chain	**Medium**	Exact version pins, frozen lockfile, CI on every change.	No automated vulnerability scan yet.
R17	Database lost	**Low**	The chain is the source of truth for authority and funds; scheduled backups in the Compose stack.	Workflow history would be lost (Render's free database expires).

4 Evidence and next steps

Evidence	What it covers
Contract tests (32, Foundry)	Owner-only changes, caps, lockup, replay, arbitrary-call denial, recovery; randomized invariants for budgets and principal.
API and worker integration (24, 29)	Sessions, tokens, owner isolation; leases, crash recovery, concurrency.
Browser suites (53 mocked, 2 real)	Every owner transaction, and the full journey with nothing mocked.
Base Sepolia run	Real owner and agent transactions; see the execution evidence report.
Public source verification	Every deployed contract matches the repository's source, checked by independent explorers (below).

4.1 Verified contract source

Each deployed contract's source was published and verified against its on-chain bytecode, so anyone can read exactly what runs at the addresses in the reviewed manifest. This backs the bytecode comparison done before the manifest was pinned, and the mitigation of R4.

Contract	Address	Verified on
MockUSDC	`0x13e7a4d02941a029Cf841f16E38eF377cb21e1Cf`	Blockscout
MockUSDCFaucet	`0xEbdc09B132066fF067E30FE077A58E818801FA2f`	Blockscout
LiquidVault	`0x7C0fC1a0551d5099d093E8D8b34a214758f7B27a`	Blockscout
TimedVault	`0x7ffD08b7b52E26023142060D3f5dBDe27757CD84`	Sourcify, exact match
BoundraFactory	`0xFbD91368BfFA9AF5C7034bCB7F53Cc1eC906b7f4`	Sourcify, exact match; Blockscout
BoundraExecutor (the evidence run's)	`0x0e4c01b7fAc2e45a2a38eA7B9A0744D4eF0e9E2E`	Sourcify, exact match

An exact match means the deployed bytecode equals the compiled source including its compiler metadata. Verified with Foundry (`forge verify-contract`), Solidity 0.8.30.

> **Before any real funds.** An independent audit of the contracts, a second RPC source, fee replacement for stuck transactions, real vault integrations with their own review, monitoring and alerts, and a legal review.